

THRONEBOUND PROJECT REPORT

A Java / JavaFX Multiplayer Strategy Card Game

CS-212 Object-Oriented Programming

NUST SEECS • Summer 2026

Technical, Functional and Implementation Documentation

Group Members & Work Distribution

ThroneBound was developed as a three-member group. The work was divided by major system areas so that each member could take responsibility for a core part of the application while coordinating during integration and testing.

Member	Roll Number	Primary Responsibility
Saad Ahmed	470964	GUI & Documentation — Group Leader
Huzaifa Ahadi	534539	Network & Persistence
Muhammad Abdur Rehman	552431	Engine & Cards

Saad Ahmed — Group Leader

Saad was responsible for the JavaFX graphical user interface and project documentation. His GUI work covers navigation, menus, deck building and the match interface. As group leader, he also coordinated the overall project and documentation.

- Designed and implemented the JavaFX user interface.
- Worked on screen navigation and visual presentation.
- Connected GUI actions with game and network functionality.
- Prepared and maintained project documentation.
- Coordinated the group and helped integrate the modules.

Huzaifa Ahadi — Network & Persistence

Huzaifa was responsible for multiplayer communication and persistent player information, including socket-based client/server communication and JSON-based persistence.

- Worked on GameClient, GameServer and client-handling logic.
- Worked on network message classes and state synchronization.
- Handled client-server multiplayer communication.
- Worked on profile, login and persistent data functionality.
- Worked with JSON storage, match history and unlock persistence.

Muhammad Abdur Rehman — Engine & Cards

Muhammad was responsible for the core game logic and card system. These parts define how turns, resources, lanes, combat, cards and victory conditions behave.

- Worked on the game engine and GameState.

- Worked on turn management, lanes, combat and resource handling.
- Implemented standard and timed win-condition logic.
- Worked on Card, Cost, Deck and Faction structures.
- Developed faction-specific card content and card interactions.

How the Three Areas Work Together

The engine provides the rules, the network layer shares those rules between players, persistence remembers player progress, and the GUI gives the player a way to interact with everything. Integration between these areas was therefore essential.

Table of Contents

1. Project Introduction
 2. Project Objectives
 3. What the Game Actually Does
 4. Game Mechanics Explained
 5. Technology Stack
 6. Repository and Package Structure
 7. Overall Software Architecture
 8. Cards Module — How the Card System Works
 9. Engine Module — How the Game Rules Work
 10. Network Module — How Two Players Communicate
 11. Persistence Module — How Player Data Is Saved
 12. GUI Module — How the Player Uses the Game
 13. Complete Application Flow
 14. Testing and TDD
 15. Build and Run Instructions
 16. Current Development Status
 17. Challenges and Design Decisions
 18. Future Improvements
 19. Conclusion
- Appendix A — Important Classes

1. Project Introduction

ThroneBound is a turn-based multiplayer strategy card game developed in Java. The project combines a fantasy card game with the software concepts expected from an object-oriented programming course. Instead of treating the game as one large program, the system is divided into separate modules for cards, game rules, networking, persistent storage, and the graphical interface.

The game is designed for two players. Each player chooses a faction, builds a deck, manages three different resources, and plays cards across three lanes. The players take turns, and the game engine decides whether their actions are legal and what effect those actions have on the board.

A major strength of the project is that the game supports LAN multiplayer using Java TCP sockets, while the persistence module allows player information and progress to survive between sessions.

In Simple Words

Think of ThroneBound as four things working together: a card game, a rule engine, a multiplayer system, and a desktop application. The GUI is what the player sees, the engine is what makes the game think, the network lets two computers communicate, and persistence makes the application remember the player.

2. Project Objectives

- Create a complete two-player strategy card game using Java.
- Apply object-oriented programming through classes, interfaces, inheritance, composition and modular design.
- Build a reusable card/faction system instead of hard-coding every card into the game.
- Implement a clear three-lane combat system.
- Use three different resources to make card costs part of the strategy.
- Support multiplayer through a client-server TCP socket architecture.
- Store profiles, match history and unlocks using persistent JSON files.
- Provide a JavaFX interface that makes the game understandable and usable.
- Use automated testing and a TDD-style workflow for important game and infrastructure logic.

3. What the Game Actually Does

3.1 Starting the Game

When the application starts, the player enters through the JavaFX interface. A player can register a new profile or log into an existing profile. After login, the main menu becomes the starting point for the rest of the application.

3.2 Choosing How to Play

The player can move toward multiplayer setup or deck construction. In multiplayer, one player can host while the other joins. The connection is established through the network module rather than being handled directly by the GUI.

3.3 Building a Deck

The deck builder lets the player work with a faction and select cards. Because the cards package contains reusable card and deck abstractions, the deck-building logic does not need to know the internal details of combat.

3.4 Playing a Match

Once the match starts, the engine becomes the main authority over what can happen. Players use cards, pay their costs, affect lanes, and take turns. In multiplayer mode, a player's action is sent to the server. The server processes it and sends the updated state back.

3.5 Finishing a Match

The game checks its selected victory condition. Standard mode follows the normal game-ending rules, while timed mode limits the match to 30 turns and uses a tiebreaker when required. After the match, the result can be stored in the player's history.

Stage	What the player experiences
1	Login or registration
2	Main menu
3	Host / join or choose deck-building
4	Select faction and prepare deck
5	Play cards and manage resources across three lanes
6	Game engine checks turns, combat and victory
7	Results and persistent match history

4. Game Mechanics Explained

4.1 Three Factions

Faction	Simple explanation
Beastkin Clans	Aggressive creature-focused identity built around pack-style synergy.
Arcane Order	Spell-oriented identity with effects that manipulate the board and resources.
Undead Legion	Death/sacrifice-oriented identity that can make use of creatures being destroyed.

The important idea is asymmetry: the factions are meant to play differently. A player should not simply choose a different visual theme and use the same strategy.

4.2 Three Resources

Cards do not all use a single universal currency. ThroneBound models Essence, Mana and Soul. A player therefore has to consider both the card's effect and whether the required resources are available.

- ResourcePool is responsible for resource accounting and affordability.
- The engine uses resource checks so invalid plays are rejected.
- Keeping resource rules in the engine prevents the GUI from becoming the authority on game legality.

4.3 Three Lanes

The battlefield is divided into three lanes. Each lane has its own state, so where a creature or effect is placed can matter. The lane structure also makes the board easier to reason about programmatically.

4.4 Turns

TurnManager keeps track of the active player and moves the match from one turn to the next. This keeps turn logic in one place rather than allowing individual buttons or network messages to decide whose turn it is.

4.5 Victory

Mode	Explanation
Standard	Uses the normal game-ending condition.

Mode	Explanation
Timed	Stops the match at 30 turns and applies the defined tiebreaking logic.

5. Technology Stack

Technology	Why it is used
Java 17+	Main programming language and OOP implementation.
JavaFX 21	Desktop graphical user interface.
Maven	Dependency management, testing and build automation.
Java TCP Sockets	LAN communication between players.
Threads / synchronized access	Client handling while protecting shared match state.
Gson / JSON	Persistent profile and game-related data storage.
JUnit 5	Automated testing.
Git / GitHub	Version control and collaboration.

Each technology has a specific role: Java runs the application, JavaFX presents it, sockets connect players, JSON stores information, and JUnit checks important logic.

6. Repository and Package Structure

```

ThroneBound/ ├── src/main/java/ | ├── cards/ | ├── engine/ | ├── network/ | ├── persistence/
|   ├── gui/ |── src/test/java/ | ├── cards/ | ├── engine/ | ├── network/ | └── persistence/
|── docs/ |── profiles/ |── pom.xml |── README.md

```

Each package represents a different responsibility. Source code is separated from tests, and the documentation folder contains supporting material such as the UML class diagram and card list.

Package	Easy explanation
cards	What cards exist and how decks/factions are represented.
engine	What the rules of the game are and how the match changes.
network	How players' computers communicate.
persistence	How player information is saved.

Package	Easy explanation
gui	How the player sees and controls the application.

7. Overall Software Architecture

The GUI receives player actions, the engine decides what those actions mean, the network distributes those actions in multiplayer, and persistence stores information that should remain after the program closes.

Layer	Main responsibility	Example
GUI	User interaction and display	MatchScreen
Engine	Rules and authoritative game state	GameState / CombatResolver
Cards	Game content and deck structures	Card / Deck / Faction
Network	Client-server communication	GameClient / GameServer
Persistence	Long-term player data	ProfileManager / JsonUtil

Why This Architecture Is Useful

If the team changes the color of a button, that should affect the GUI, not the combat engine. If the team changes how a card's cost is calculated, that should be handled by the game rules, not the socket code. This separation makes the project easier to maintain.

8. Cards Module — How the Card System Works

The cards module defines the objects that make the game a card game. Instead of representing every card with unrelated code, the project provides common abstractions that can be reused.

Main Responsibilities

- Card.java provides the base card abstraction.
- Cost.java represents the resources required to play a card.
- Deck.java manages a collection of cards and deck/draw behavior.
- Faction.java provides a common faction abstraction.
- Faction-specific classes and card pools provide different identities and content.
- CardFactory provides a convenient way to create card objects.

Why This Is Good OOP

A common Card abstraction allows different cards to be treated as cards by the rest of the system while still allowing their individual effects to differ. The same idea applies to factions.

9. Engine Module — How the Game Rules Work

The engine is the heart of ThroneBound. If the GUI is the part the player sees, the engine is the part that answers questions such as: Is this card affordable? Whose turn is it? What happens when cards interact? Has the game ended?

GameState

GameState represents the current match, including the board situation, players, resources and game progress.

Lane

Lane represents one of the three battlefield positions. Keeping lane state in its own class makes the board easier to manage.

CombatResolver

CombatResolver contains the logic for resolving interactions so combat rules remain consistent regardless of where an action originated.

ResourcePool

ResourcePool tracks resources and checks whether a player can afford a cost.

TurnManager

TurnManager controls turn transitions and the active player.

WinCondition

WinCondition is an interface that allows different game-ending strategies. StandardWinCondition and TimedWinCondition provide implementations.

Typical Card-Play Decision

Player chooses card ↓ GameClient / GUI sends action ↓ Engine checks active player ↓ Engine checks card cost ↓ Engine applies card effect ↓ GameState changes ↓ Updated state is shown to player(s)

10. Network Module — How Two Players Communicate

The multiplayer system follows a client-server design. One computer hosts the game server and the other connects as a client. The server processes actions and keeps the match state authoritative.

Class	Role in simple terms
GameServer	Waits for players and hosts the match.
ClientHandler	Handles messages from a connected client.
GameClient	Connects to the server and sends player actions.
Message	Base type for network messages.
SetupMessage	Carries match setup information.
PlayCardMessage	Requests a card play.
EndTurnMessage	Requests the end of a turn.
StateUpdateMessage	Carries updated game state to clients.
GameStateSnapshot	Serializable version of state sent over the network.

Why the Server Is Authoritative

The client does not decide whether its own action is legal. It sends a request. The server passes the action into the game logic, and the resulting state is sent back. This gives both players one source of truth.

Concurrency

The server handles connected clients using threads. Because multiple threads may interact with shared match data, synchronized access protects the game state. The test suite also includes a concurrency test for playing cards simultaneously.

11. Persistence Module — How Player Data Is Saved

Persistence answers a simple question: what should the application remember after it closes? ThroneBound stores player information using JSON.

Stored Information

- Player identity and login-related profile information.

- Win/loss and match-history information.
- Unlock information.
- Saved deck/progression information described by the project.

Main Classes

Class	Purpose
PlayerProfile	Represents a player's persistent information.
ProfileManager	Registers, logs in, loads and saves profiles.
MatchHistory	Keeps a collection of match results.
MatchRecord	Represents one completed match.
UnlockManager	Handles persistent unlock information.
JsonUtil	Reads and writes JSON data.
InvalidCredentialsException	Represents unsuccessful login validation.

Persistence tests use temporary directories so test files do not interfere with real user data. This keeps automated testing isolated and repeatable.

12. GUI Module — How the Player Uses the Game

The GUI translates the technical game system into something a player can understand. JavaFX is used for the desktop interface.

Screen	Purpose
LoginScreen	Register or log into a player profile.
MainMenuScreen	Central dashboard for navigation.
HostJoinScreen	Choose host or join mode.
DeckBuilderScreen	Select faction and prepare a deck.
MatchScreen	View lanes, cards, player information and actions.
ResultsScreen	View the outcome after the match.
ThroneBoundApp	Starts the JavaFX application and initializes the scene.
SceneManager	Controls navigation by changing the current root.

Single-Scene Navigation

The application is designed around one shared JavaFX Scene. Instead of repeatedly creating new stages and scenes, SceneManager replaces the root node. This makes the window and common styling more consistent.

Current UI Direction

The current UI work emphasizes a fantasy-themed purple palette, consistent backgrounds, better panel contrast, centered host/join controls, and more polished card and deck-builder layouts.

13. Complete Application Flow

LOGIN ↓ MAIN MENU | — Host / Join | ↓ | Deck / Match Setup | ↓ | — Deck Builder ↓ MATCH SCREEN ↓ GAME ENGINE ↓ RESULTS SCREEN ↓ PERSISTENT HISTORY

In multiplayer mode, MatchScreen communicates with GameClient, which communicates with the server. The server uses the engine to update the authoritative GameState and then sends the new state back to clients.

Networked Match Flow

Player A GUI → GameClient → TCP Socket → GameServer ↓ Game Engine ↓ GameState ↓ StateUpdateMessage ↓ Client A / Client B GUI

14. Testing and TDD

Testing is a major part of the project rather than something added only at the end. The repository documents a TDD-style process in which tests are prepared from the UML design, implementations begin as incomplete methods, and the code is developed until the tests pass.

Red → Green → Refactor

Stage	What it means in ThroneBound
Red	A test exists for behavior that has not been implemented yet.
Green	The implementation is added until the test passes.
Refactor	The code can be cleaned up while tests continue to pass.

Important Tests

Area	Examples
Engine	GameStateTest, LaneTest, ResourcePoolTest, TurnManagerTest, CombatResolverTest
Cards	CardTest, FactionTest, DeckTest, BeastkinCardTest
Network	MessageTest, NetworkIntegrationTest
Persistence	PlayerProfileTest, ProfileManagerTest
GUI	No dedicated automated GUI test package is listed; GUI behavior is mainly verified manually.

14.1 MessageTest — Network Message Validation

The uploaded evidence shows the MessageTest class and its successful execution in IntelliJ IDEA. The test verifies that different network message objects carry the expected type and payload information. This is important because multiplayer communication depends on messages being correctly identified and carrying the correct data.

- PlayCardMessage carries the card name and lane index. The screenshot shows the test using the card name “Wolf Pup” and lane index 1.
- EndTurnMessage reports the expected message type “END_TURN”.
- StateUpdateMessage reports “STATE_UPDATE” and preserves the JSON payload.
- The test run completed successfully with 3 tests passed and exit code 0.

MessageTest Source Code

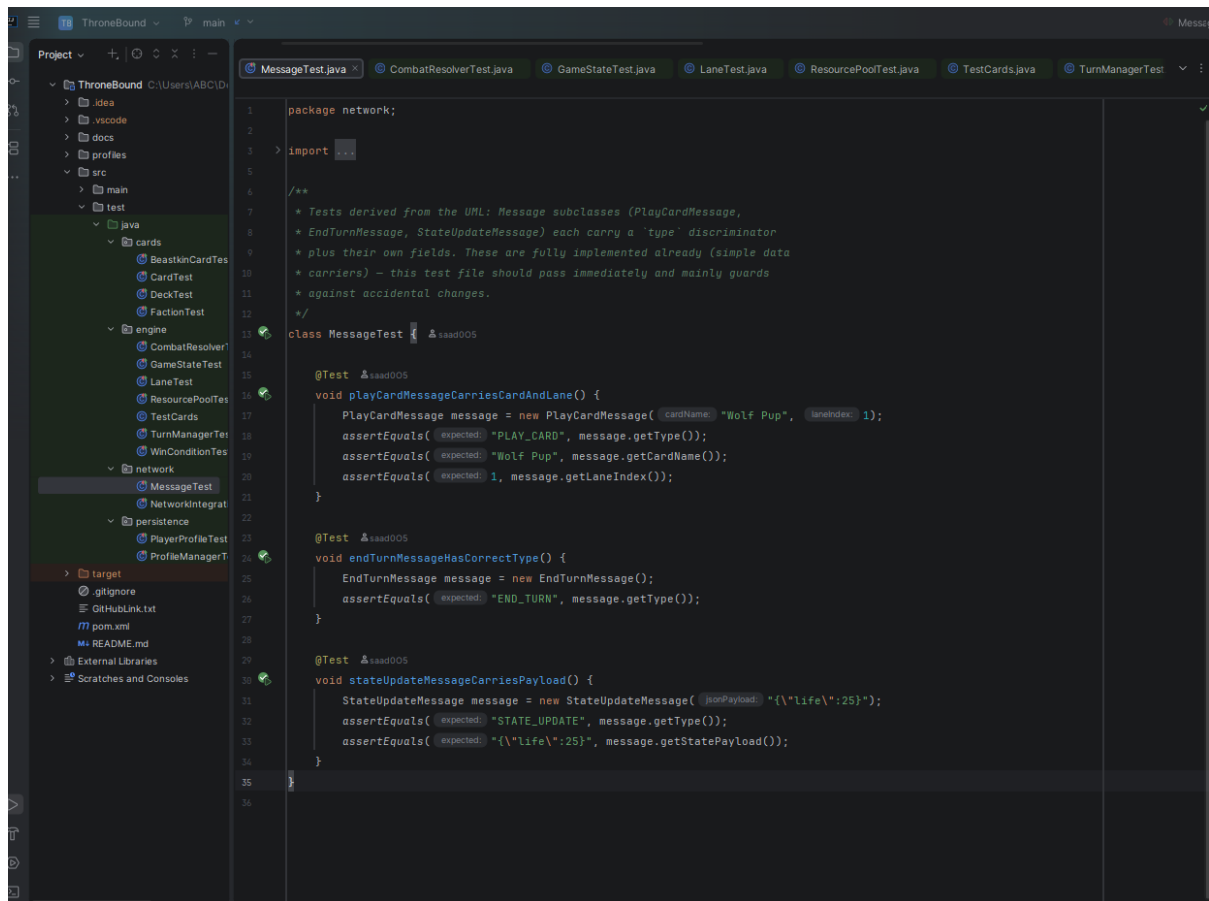


Figure 1. MessageTest source code showing the three network-message tests.

MessageTest Console Output

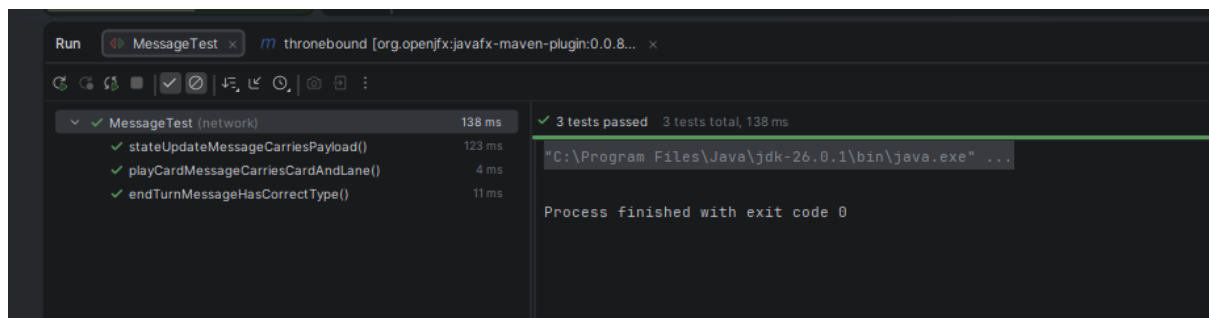


Figure 2. IntelliJ test output showing all 3 MessageTest tests passed.

Result: The evidence confirms that the MessageTest test class executes successfully. IntelliJ reports “3 tests passed” and the Java process finishes with exit code 0. This provides direct validation of the message layer behavior described in the network module.

15. Build and Run Instructions

15.1 Requirements

- JDK 17 or newer
- JavaFX 21

- Maven
- Internet access for Maven dependencies

15.2 Run Tests

```
mvn test
```

Runs the JUnit test suite.

15.3 Run the JavaFX Application

```
mvn javafx:run
```

Launches the JavaFX application using the project's Maven JavaFX configuration.

15.4 Package the Project

```
mvn package
```

Builds the project through Maven.

16. Current Development Status

Completed / Established Areas

- Core card and faction architecture.
- Three-lane game structure.
- Three-resource economy.
- Turn and combat engine.
- Standard and timed victory conditions.
- Client-server networking structure.
- Persistent JSON profiles and match information.
- JavaFX screens for the major user journey.
- Automated tests for engine, cards, network and persistence.
- TDD-oriented development process.
- Single-scene navigation and fantasy UI styling work.

Areas Still Worth Validating

- Full host-to-client match flow should be tested end-to-end.
- GUI interactions should continue to be manually checked because automated GUI tests are limited.
- Card affordability indicators should clearly show when a card cannot be played.
- Layout and contrast can continue to be polished.

- Disconnect and invalid-action scenarios should be tested carefully.

17. Challenges and Design Decisions

Keeping the Rules Out of the GUI

A common problem in game projects is putting too much logic inside button handlers. ThroneBound avoids this by keeping game state and rule processing in the engine.

Keeping Multiplayer Consistent

Two players must see the same game. Making the server authoritative and transferring state snapshots gives the system a central source of truth.

Handling Shared State

Multiple client-handling threads can access the same match. Synchronized access and concurrency testing are therefore important.

Keeping Player Data Separate

Persistent data should survive normal application use but must not contaminate automated tests. JSON persistence combined with temporary test directories gives the project a simple solution.

18. Future Improvements

- More content: Add additional factions, cards and abilities using the existing card/faction structure.
- Better GUI testing: Add automated tests for critical navigation and interaction paths.
- Multiplayer resilience: Improve handling of disconnects, reconnects, invalid messages and server shutdown.
- Balance testing: Collect match results and use them to tune faction/card balance.
- Persistence robustness: Add stronger validation and migration support if the JSON format changes.
- Visual polish: Improve the match board, deck builder, card readability and responsive layout.
- Player feedback: Add clearer messages for invalid moves, insufficient resources and connection problems.

19. Conclusion

ThroneBound demonstrates how a relatively simple game idea can become a complete software engineering project. The player sees cards, lanes and buttons, but underneath those screens are several coordinated systems.

Muhammad's engine and cards work provide the rules and content of the game. Huzaifa's network and persistence work allow players to communicate and keep their progress. Saad's GUI and documentation work turn the underlying system into a usable application and keep the project understandable. The three areas depend on one another, making integration an important part of the project.

From an OOP perspective, the project demonstrates abstraction through interfaces and base classes, modularity through packages, composition through relationships between game objects, and practical use of testing, networking, concurrency and persistence. This makes ThroneBound a complete example of how multiple software components can be combined into one working Java application.

Appendix A — Important Classes

Module	Classes to remember for presentation/viva
Cards	Card, Cost, Deck, Faction, faction-specific classes, CardFactory
Engine	GameState, Lane, ResourcePool, TurnManager, CombatResolver, WinCondition
Network	GameServer, GameClient, ClientHandler, Message classes, GameStateSnapshot
Persistence	PlayerProfile, ProfileManager, MatchHistory, MatchRecord, UnlockManager, JsonUtil
GUI	ThroneBoundApp, SceneManager, LoginScreen, MainMenuScreen, HostJoinScreen, DeckBuilderScreen, MatchScreen, ResultsScreen